

固定调度机制下被抢占请求的原生恢复方式选择

技术报告初稿：问题、实现边界与正常低负载观测

独立研究会话 E

2026 年 10 月 7 日

摘要

在支持 Host KV 卸载的推理服务中，一个被抢占请求再次获得恢复机会时，可以加载已经保存的合法 KV，也可以利用原生重算路径重建所需状态。本文考察固定恢复目标、victim 规则、队列、服务量子及准入后，仅改变这一次执行方式的效果。核心问题是：长度相近的恢复请求在不同系统状态下，两种动作的完整服务代价是否不同；若存在差异，决策前可观察的一个状态信号是否能提供超出长度或孤立成本的增量信息。当前已完成 vLLM 原生路径的薄适配及首个正常容量 64 请求对照。Host/Recompute 两臂合计完成 128 次请求执行，64 对输出 token 序列全同；实际 GPU KV 容量为 72.1015625 GiB，观测峰值占用仅 20.44%，两臂均无抢占或恢复动作。因此，实测服务时间差异不能归因于恢复方式，恢复选择收益仍未检验。文中保留已支付 STORE 的完整成本口径，并列明后续容量与合法恢复事件验证。

1 问题与可检验范围

设请求 r 在时刻 t 已收到的输入为 P_r ，已经对外产生的 token 为 $Y_r^{\leq t}$ 。恢复必须保留的已知历史为

$$H_r(t) = P_r \parallel Y_r^{\leq t}, \quad n_r(t) = |H_r(t)|. \quad (1)$$

选择器只读取当前和历史观测，不能使用未来输出长度，也不能先执行两条路径再选择较快结果。本文考虑如下两个动作：

- $a = \text{Host}$ ：加载 Host 中已经就绪、与当前历史一致的原生 KV 前缀，并由原生路径补齐未命中的必要尾部。
- $a = \text{Recompute}$ ：保留同一个请求及其已知 token 历史，经原生调度器和模型执行器重建必要 KV，随后继续生成。

Host 命中并不要求覆盖全部历史；两条路径最终都必须到达原请求下一次合法生成所需的完整状态。研究对象不是重新提交一个新请求，也不是把已经生成的文本当作新的回答重新采样。

第一版固定恢复目标、抢占对象选择、FCFS 队列、token 预算、准入检查和传输实现。动作导致后续系统状态自然变化是实验结果的一部分；不能为了强行对齐两臂的执行轨迹，再修改调度器。对照固定的是外部到达序列、资源预算和机制参数，而不是要求两次运行具有完全相同的抢占事件集合。

待检验假设是：孤立恢复速度可能没有反映传输排队、GPU 计算竞争及其他请求的停顿。不过，这只是实验动机；本文不预设存在交叉点，不预设状态感知规则优于最佳固定动作。若当前运行域始终由一个动作占优，或只有一条路径可合法执行，应直接报告该边界。

2 成本口径与正确性边界

令 t_r^{next} 为一次已提交恢复选择之后该请求首次新输出的时间。局部观测为

$$L_r^{\text{next}} = t_r^{\text{next}} - t_r^{\text{decision}}. \quad (2)$$

该时间包含真实排队、传输、重算和调度等待，不能用孤立复制时间替代。若请求在下一输出前再次被抢占，多次选择可能对应同一次输出；分析器保留这种关联，不将这些记录当作独立因果样本。

全局观测使用所有请求的外部到达 A_i 、完成 C_i 以及连续输出时间 $O_{i,k}$ ，至少报告平均完成时延、整体完工时间以及生成间隔：

$$\bar{F} = \frac{1}{N} \sum_{i=1}^N (C_i - A_i), \quad M = \max_i C_i - \min_i A_i, \quad G_{i,k} = O_{i,k} - O_{i,k-1} \ (k > 1). \quad (3)$$

同时报告 TTFT、每请求最大停顿、间隔分位数和未完成请求。正常容量主组保留自然 EOS，预先声明的输出上限只限定合法生成范围；必须同时报告实际输出长度、EOS/长度截断数量和输出差异，不能假设各臂工作量恒等。固定输出只用于另行标记的等工作量诊断。自然停止本身也不构成任务质量或生产 goodput 证据。

已经发生的 STORE 是沉没成本。 在 t 之前，为请求保存 KV 的计算与传输已经真实发生。无论随后选择哪个动作，该历史成本都保留在整段服务统计中；比较当前动作的增量影响时，不能将其再次计入 Host 分支，也不能将其记作重算节省。两臂均保留同一原生 STORE 机制。动作以后新增的计算、LOAD、STORE 和排队仍由真实执行统计。异步传输与计算可能重叠，CUDA 事件时间可用于诊断，却不能简单相加来重建端到端墙钟时间。

完整状态容量。 设每个 GPU KV block 容纳 b 个 token，当前已知历史至少需要

$$B_r = \lceil n_r(t)/b \rceil. \quad (4)$$

当前薄适配器仅在 $B_r + B_{\text{reserved}} + B_{\text{watermark}} \leq B_{\text{free}}$ 时允许改变原生结果，并继续执行原生分配器的全部检查。运行器打开完整序列容量约束。该条件不是声称只加载较短 Host 前缀便可减少完整状态所需容量，也不替代模型执行的其他内存要求。

请求连续性。 选择前后核对对象身份、全部已知 token、采样参数对象、输出进度、状态及停止原因，记录前缀哈希。输出必须单调扩展已有序列，重建未完成时不得冒出新输出 token。首轮使用 greedy 路径；不据此宣称已经验证随机采样的 RNG 状态等价。当前完成的低负载组没有执行恢复动作，因此其输出一致性也不能代替恢复状态不变量的实测验证。即使历史完全相同，批形状改变也可能引入数值差异，故跨臂 token 差异应如实保留，并在后续自然任务中单独评估质量。

3 原生接口与薄适配

实现依据本地保存的 vLLM 0.26.0 源码快照。原生 connector scheduler 的 `get_num_new_matched_tokens` 返回可用外部前缀；随后 `update_state_after_alloc` 在 GPU 分配后准备加载，并调用 `manager.prepare_load`。原生主调度器的 `_preempt_request` 重置已计算进度，请求对象仍持有 `all_token_ids`，供调度器恢复计算。源文件路径和哈希由项目的 `native_sources/` 与 `source_manifest.json` 保存。

`selector.py` 在原生 lookup 之后作一次小范围适配。只有已经抢占过、存在正的就绪 Host 命中且满足共同容量条件的请求进入可选集合。Host 策略原样保留命中；Recompute 策略将 connector 的命中元数据同步归零并返回 `(0, False)`，让同一个原生请求进入正常计算路径。选择器不分配另一套 KV，不实现新的重算执行器，也不在选择前执行传输或模型计算。双方都经过原生 lookup，保留其共同的查找行为。

lookup 可能被尝试多次而未真正获得资源，因此尝试次数不等于执行决策数。当前记录以 `update_state_after_alloc` 为提交边界，分别保存偏好、返回结果与实际外部 token 数。尚在等待本请求原生传输时沿用原生延期；Host 未命中时沿用原生重算；共同容量不足时保留原生结果与准入检查。这些 fallback 独立计数，不作为选择器成功样本隐藏。

适配依赖私有字段，故版本变化可能破坏假设。运行时限定原生 CPU offloading、单 KV group、非滑窗及关闭 APC，并检查预热后缓存重置、实际 GPU/Host 存储字节数和 Host pinned 属性。断言失败应留下失败记录并停止该实验格。当前可运行策略为固定 Host、固定 Recompute 及显式长度阈值；尚未加入声称有效的动态规则。目前验证范围是适配器结构审计及正常低负载、无恢复动作的 GPU 服务链通过，Host/Recompute 恢复分支仍未由合法事件触发，不能称为两种恢复方式均通过 GPU 正确性验证。

4 首轮实验协议

表 1 给出面向 PRO 6000 正常容量的统一协议；64 请求组已完成，其实测结果见第 7 节，后两档尚无测量。GPU KV 采用 `gpu_memory_utilization=0.9` 下的原生 profiling，不设置显式 KV 字节上限；记录实际可用 block 数和张量存储字节数。每个实验组只初始化并 profile 一个引擎，各臂复用该引擎，保证组内实际容量一致。不同实验组释放 GPU 后重新初始化，必须分别报告实际容量，不以相同 utilization 参数代替容量核对。原 16 请求、1 GiB GPU KV 配置不再是主实验；显式限容仅可作为独立标记的诊断。

表 1: 正常容量首轮配置；最终以每组日志及资源断言为准。

项目	配置
设备与模型	RTX PRO 6000 Blackwell Server Edition；服务器已有 OLMoE 本地快照
软件与执行	vLLM 0.26.0；PyTorch 2.11.0；Transformers 5.17.0；BF16、eager、原生 Triton MoE
KV 预算	GPU 原生 0.9 profiling；pinned Host 32 GiB；记录 chunk 舍入后实际容量；APC、推测解码关闭
调度与准入	FCFS；最多 256 请求；每步 2048 token；完整当前历史必须可容纳
工作负载	64、192、256 请求三档；固定 1 短:3 长；相同档位各臂输入和外部到达相同，外部到达均为 0
输入与输出上限	短 GSM8K 输入 661–约 800 token、输出上限 256；长完整 QA 输入 2699–3271 token、输出上限 512
生成与顺序	greedy、自然 EOS，256/512 为上限；低负载和接近容量候选各 Host/Recompute 两臂，高压候选用 Host/Recompute/Recompute/Host
引擎与隔离	同组各臂复用同一引擎；每臂前相同完整预热和缓存重置；组间释放 GPU

10 月 7 日恢复访问后的实测 GPU 身份为 GPU-94203fc3-1021-3a9c-a367-cff792479616。这是本次测量的新设备标识；不将早期入口的设备状态或未运行计划视作本组测量。

工作负载由 `inputs_pro6000/freeze_receipt.json` 在 CPU 侧冻结。短提示来自公开 GSM8K test 的 64 个选定样本及既有八示例前缀 [8]；长提示来自 LongBench `multifieldqa_en` 的 17 条完整自然 QA [9]，包含原上下文和问题，不通过截断或重复拼接文本构成长输入。长任务循环复用这些样本，但每次提交是独立请求并使用独立缓存身份；关闭 APC，不能借重复提示获得 GPU 前缀复用。三档分别含 33、65、81 个不同提示，是重复自然请求的压力探针，不能作为独立任务泛化或质量结论。

表 2: 冻结的三档负载。容量列仅是假设所有请求同时驻留且均生成到上限时的完整 KV 上界估计，非实测分配或抢占证据。

候选档位	请求数	短/长请求	不同提示数	上界估计 (GiB)
低负载	64	16/48	33	22.35
接近容量	192	48/144	65	67.23
高压	256	64/192	81	89.66

表 2 的估计按 16-token block 舍入，来自冻结清单；实际驻留时间、原生准入与完成顺序可能使峰值占用显著低于此值。“接近容量”和“高压”是候选负载标签，须由实际 block 使用、队列、抢占与合法 Host 命中记录确认。Host 预算也不保证恢复命中，原生驻留和淘汰决定可用前缀。

同组每臂开始前，先排空上一 episode 的原生传输并重置 Host/GPU 缓存；再关闭选择器，用该档全部请求执行相同原生 Host 预热，排空 STORE/LOAD 后再次重置。当前共同预热仍忽略 EOS 并生成到各自上限，用于覆盖并发批形状；这些输出不属于正式测量。复位后正式主组使用自然 EOS，不能把预热期间的抢占当作主组恢复。检查请求队列、connector 作业与缓存空闲基线，防止前臂状态泄漏。计时从重置后的正式 episode 开始；版本、配置、输入清单、预热成本、资源、状态和原始事件均单独落盘。若预热成本需要缩减，应依据运行证据另行冻结相同的新预热方案，不在组内改动。不能事后删除不利运行；超时或断言失败时保留部分轨迹

与失败原因。

每个有限实验组持有现有共享 GPU 的共同整组锁，初始化和每臂前检查设备身份与外部计算进程。按低负载两臂、接近容量两臂、高压 ABBA 的顺序推进；每组结束后退出引擎并释放 GPU，不维持 command loop 或在分析期间占锁。未在限定候卡时间内取得锁则记录中止，不终止其他进程，也不在其他实验的格间空隙插入作业。该规则属于资源协调，不是实验性能数据。

`run_cell.py` 记录外部到达与实际入队时间、每次原生调度的计算区间、抢占、选择/提交、每个新输出及完成时刻，以及原生 LOAD/STORE 报告。输出时刻取本地同步 `engine.step()` 返回后的观测，包含 Python 驱动与插桩开销，尚不是网络流式客户端时延。`analyze.py` 汇总上述局部与全局指标，并区分 lookup 尝试、提交和 fallback。所有统计以真实服务为基础，不离线扣除估计开销构造另一个动作。

低负载首先检验正常容量基线与追踪完整性；高压 ABBA 再检查方向的反序稳定性。正常容量下没有抢占或没有两路径均合法的恢复事件也是有效结果，恢复时延应记为不适用，不能填零后宣传收益。一次突发和重复自然提示不足以支持稳定尾分位数或统计显著性论断。两臂的事件数可能不同；不得将跨运行、同长度但不同系统历史的恢复事件直接视为随机配对。可对照的是外部请求及整段服务，事件分组只作为解释性诊断。

5 最小在线规则与后续推进条件

若真实对照出现可重复且值得利用的差异，先增加长度/成本阈值

$$\pi_{\ell}(n) = \begin{cases} \text{Recompute}, & n \leq \tau, \\ \text{Host}, & n > \tau, \end{cases} \quad (5)$$

并用独立校准运行确定 τ ，随后固定到验证运行。若测量不支持这一方向，应调整或弃用该假设，而不是强制制造阈值交叉。硬件成本基线只比较未来的 LOAD/必要尾部计算与重建成本。

当前轨迹同时留有待处理 LOAD 数、队列规模和近期原生步骤时间，目的是诊断，不代表在线规则应同时使用所有特征。下一次有针对性的改动只选择一个有证据的信号 z_t ：例如已暴露的传输等待，或上一步计算耗时。可检验的最小规则是在原有阈值上设置一个状态分支，但分支方向及阈值必须由先前数据决定，在验证序列到达前冻结。禁止使用未来输出长度，禁止将事后最优动作算作在线收益。

比较对象必须包含最佳固定方式、固定长度/成本阈值和增加一个状态信号的规则。若规则退化为固定阈值，应直接报告等价性；若状态在控制长度后没有增量解释力，应删除该状态模型。若局部恢复变快而全局完成或生成间隔恶化，下一步先检查该动作占用了哪些请求的执行机会，而不是继续扩大参数搜索。

信号稳定后再扩展到独立任务、不同到达形状与至少一个独立负载；报告任务质量、完整开销及所有失败。需要另行测量有无插桩的差异、选择器运行开销与随机采样连续性。主组 GSM8K 与长 QA 虽保留自然 EOS，17 条长 QA 的循环独立提交仍不增加不同任务数量，不能直接提供泛化或问答质量结论。`--fixed-output` 仅用于明确标记的等工作量诊断，不能用其强制生成的压力替代自然主组证据。

6 相关工作与研究定位

CacheOPT 按模型和硬件对长度建立重算、swap 成本函数，其比较包含 swap-out 和 swap-in；系统还联合改变其他 KV 分配和调度机制 [1]。Stream2LLM 在 preemption 时采用硬件成本选择，其实现明确使用闲置系统静态 profile，并独立比较调度与驱逐策略 [2]。这两者是成本基线来源；在本文 STORE 已支付的边界上必须重新标定。Stream2LLM 官方源码函数尚未成功访问，当前仅核对论文与 artifact 说明，不宣称已经复现其实现。

UnboxKV 已实测并发、GPU KV 容量与模型影响 swap/recompute 排名，且指出重复 prefill 会影响调度 [3]。因此，观察到负载相关排名变化本身不是本文可主张的新发现。Cake 双向并行计算前缀与加载后缀，并调整与其他请求共享算力的优先级 [4]；CacheGen 根据网络条件在不同压缩 KV 与文本重算之间适配 [5]。这些工作已经建立资源状态与计算/加载选择之间的关系，其混合执行、编码或调度变化超出本文第一版动作空间。

LUMEN 在故障恢复时根据 worker 负载，必要时放弃 checkpoint 本地性并改派请求重算 [6]。其目的地选择与本文固定目的地的动作不同，但使“首次负载感知恢复”这一表述不成立。ShadowServe 研究 KV 解压与模型计算的干扰，说明局部数据路径指标不足以替代完整服务指标 [7]；本文没有解压步骤，不能沿用其干扰量级。本文当前定位是一个受控问题与最小实现，是否具有超出强近邻的实证价值仍取决于后续真实数据。

7 首个真实服务结果与未完成验证

数据与完成状态。 execution/oct07_low_r02/summary.json 记录同引擎 Host → Recompute 的两个完整测量格；low64_results.txt 独立从原始轨迹重算并核对了主要指标。两臂各完成 64/64 请求，合计 128 次请求执行，外部到达均为 0。输入、外部负载、固定配置、引擎参数、资源及预热形状指纹匹配；两臂预热后重置均成功，待处理作业和请求为零。汇总中无分析错误、无失效格、无未完成格。输入冻结清单中的 FROZEN_CPU_ONLY 是准备阶段历史状态，不能继续代表本次执行状态。

恢复决策域未激活。 实际 GPU KV 为 72.1015625 GiB、36,916 blocks, pinned Host 为 32 GiB、16,384 blocks。两臂初始空闲 GPU blocks 均为 36,915；观测最少空闲为 29,371，峰值使用为 7,545/36,916，即 20.4383%，包括初始常占的 1 block。该占比来自 schedule/step/preempt 采样边界，不是完整 allocator 高水位，也不是物理 GPU 总显存占比。Host 常驻峰值为 10,137 blocks (61.8713%)，step 边界运行/等待请求峰值为 50/62。

两臂抢占请求、恢复选择尝试、Host-ready 事件、共同可行事件与已提交恢复动作全部为零，fallback 也没有被触发；LOAD 字节和实际传输数均为零。故 decision-to-next-output 无可估计样本，应标为不适用。该正常低负载组未提供一次合法恢复选择的实验空间，不能验证 Host 或重算的恢复速度、恢复正确性或状态信号的增量信息。

表 3: 正常 64 请求组的实测服务指标。列名仅表示所配置策略；两臂实际恢复动作均为零，差异不构成策略收益。

指标	Host 配置	Recompute 配置
完成请求数	64/64	64/64
实际输出 token 数	8,694	8,694
完工时间 (s)	9.692865	9.582993
平均完成时延 (s)	3.956571	3.978664
P95 完成时延 (s)	7.540868	7.566621
P99 生成间隔 (ms)	35.725633	35.479613
输出吞吐 (token/s)	896.948440	907.232212
服务及 drain 结束 (s)	9.692929	9.583051
已提交恢复动作	0	0
恢复到下一输出	不适用	不适用

表 3 的完成时延从固定外部到达计算，生成间隔不含 TTFT。因为实际选择器动作均为零，这些时间差异只能描述一次顺序运行的服务波动，不能归因于恢复方式，也不支持优胜策略或显著性结论。64 对请求的 greedy token 序列全部相同，完成/停止元数据无差异；每臂只有 3 个请求以长度上限结束，其余自然停止。这是本组输出一致性检查，不是 GSM8K 或长 QA 的质量评分。

保留完整测量期 STORE。 两臂各报告 STORE 21,258,829,824 bytes (19.798828 GiB)、685 次传输；CUDA-event 持续时间和分别为 0.410424 s 与 0.413987 s。统计包含测量期及 drain，不含共同预热与初始化；未从 Recompute 配置减去已支付 STORE。LOAD 的实际字节、次数和时间均为零，汇总中的 300 个 reporting records 只是包含空 LOAD 字段的元数据批次，不能当作 LOAD 次数。传输时间可与计算重叠，不能相加得到墙钟；全请求生成间隔之和 180.329289 s 与 179.644216 s 同样不是墙钟。两臂原生调度 token 数均为 162,584，重复计算位置为零，与未出现恢复相符。

未完成部分。 192 请求组 r01 因锁忙在模型加载前中止；r02 的 controller PID 4987 在本稿更新时确认为内核 `WRITE*wait`，等待上限为 1800 s，尚未进入测量，不能标记为实验运行中。256 请求组、显式限容诊断及加入当前状态信号的规则仍为 `UNRUN`。本稿是持续研究的阶段报告，当前零动作结论仅限已完成的正常 64 请求负载。后续只在获得新原始数据后回填：

1. [待回填：192/256 请求组的实际容量、占用、完成状态和恢复覆盖率；容量上界不能证明出现抢占。]
2. [待回填：存在恢复时的合法双分支事件、动作与 `fallback`、`decision-to-next-output`，以及高压力反序重复。]
3. [待回填：恢复被执行后的请求状态与输出连续性、真实传输和重复计算、对全部请求的影响及适配/插桩完整开销。]
4. [待回填：是否支持增加一个状态信号；如支持，具体信号、冻结阈值、独立验证与自然任务质量。]

下一步先检查 192 请求组是否产生真实抢占及合法双分支恢复，再决定是否推进 256 请求组。若正常容量仍无恢复，应保留这一适用域结论；只有需要定位机制时，才另设显式限容的诊断配置，并与正常硬件结果分开报告。现有零动作结果不为动态状态规则提供训练依据，也不应促使无界搜索参数以制造收益。

参考文献

- [1] Haiying Shen and Tanmoy Sen. Mitigating KV Cache Competition to Enhance User Experience in LLM Inference. arXiv:2503.13773v1, 2025. <https://arxiv.org/html/2503.13773v1>.
- [2] Rajveer Bachkaniwala et al. Stream2LLM: Overlap Context Streaming and Prefill for Reduced Time-to-First-Token. arXiv:2604.16395v1, 2026. <https://arxiv.org/html/2604.16395v1>. Official artifact: https://github.com/rajveerb/stream2llm/tree/mlsys_artifact.
- [3] Characterizing the Behavior and Impact of KV Caching on Transformer Inferences under Concurrency. Author-hosted full text, 2025. <https://jcernuda.com/publications/kv-caching>.
- [4] Shuowei Jin et al. Compute Or Load KV Cache? Why not Both? arXiv:2410.03065v2, 2025. <https://arxiv.org/html/2410.03065v2>.
- [5] Yuhao Liu et al. CacheGen: KV Cache Compression and Streaming for Fast Language Model Serving. SIGCOMM 2024; inspected arXiv:2310.07240v4. <https://arxiv.org/html/2310.07240v4>.
- [6] Zhang Cao et al. LUMEN: Coordinated Failure Recovery for Distributed LLM Serving. arXiv:2606.17787v1, 2026. <https://arxiv.org/html/2606.17787v1>.
- [7] Xingyu Xiang et al. ShadowServe: Interference-Free KV Cache Fetching for Distributed Prefix Caching. arXiv:2509.16857v1, 2025. <https://arxiv.org/html/2509.16857v1>.
- [8] OpenAI. Grade School Math, public test data. Selected test indices 128–191; file hashes preserved in the experiment freeze receipt. https://github.com/openai/grade-school-math/blob/master/grade_school_math/data/test.jsonl.
- [9] LongBench. Public multifieldqa_en data; revision 5e628be450b7e67fb7ae6e201bd6d8f7056f7672. Selected 17 source rows and hashes preserved in the experiment freeze receipt. <https://huggingface.co/datasets/zai-org/LongBench>.